

좋습니다. 1번은 LLM도, LangChain도, LangGraph도 쓰지 않고 시작하는 게 맞습니다. 먼저 “주식 분석 로직 자체”를 순수 Python 함수로 만들어야 이후에 그 함수들을 Tool로 감쌀 수 있습니다.

이번 1단계의 목표는 아주 명확하게 잡겠습니다.

종목의 OHLCV 데이터를 넣으면 이동평균·RSI·MACD·ATR·ADX·거래량을 계산하고, 규칙에 따라 기술적 점수와 BUY/HOLD/SELL 신호를 내는 Python 주식 분석기 만들기

처음에는 삼성전자 같은 한 종목으로 시작하고, 나중에 기본적 분석·수급·뉴스를 붙이는 방식으로 확장하겠습니다.

1. 프로젝트 폴더부터 만듭니다

Windows PowerShell 기준입니다.

PowerShell

```
mkdir stock-ai-agent
cd stock-ai-agent

uv init
uv venv

.venv\Scripts\activate
```

필요 패키지를 설치합니다.

PowerShell

```
uv add pandas numpy yfinance
```

여기서는 학습 목적으로 `yfinance`를 쓰겠습니다. 한국투자증권 API는 11단계에서 붙이는 게 좋습니다.

폴더는 우선 이렇게만 만듭니다.

```
stock-ai-agent/
|
├─ main.py
├─ stock_analyzer.py
└─ pyproject.toml
```

2. 가장 먼저 이해해야 하는 구조

우리가 만들 프로그램은 이것입니다.

```
종목코드
↓
주가 데이터 가져오기
↓
지표 계산
↓
규칙 평가
↓
점수 계산
↓
BUY / HOLD / SELL
```

중요한 것은 각 단계를 함수 하나씩 만드는 것입니다.

나중에는 그대로:

```
Python Function
↓
LLM Tool
↓
LangChain Tool
↓
LangGraph Node
```

로 발전합니다.

예를 들어 지금 만드는:

Python

실행됨

```
calculate_rsi()
```

라는 함수가 나중에는 LLM이 호출할 수 있는:

```
calculate_rsi Tool
```

이 됩니다.

따라서 지금 함수 설계를 잘하는 것이 중요합니다.

3. 첫 번째 파일: `stock_analyzer.py`

아래 코드를 그대로 만들어 보세요.

```
import pandas as pd
import numpy as np
import yfinance as yf

# -----
# 1. 주가 데이터 가져오기
# -----

def get_stock_data(
    ticker: str,
    period: str = "1y",
) -> pd.DataFrame:

    df = yf.download(
        ticker,
        period=period,
        auto_adjust=True,
        progress=False
    )

    return df

# -----
# 2. 이동평균
# -----

def calculate_moving_averages(
    df: pd.DataFrame
) -> pd.DataFrame:

    df = df.copy()

    df["MA5"] = df["Close"].rolling(5).mean()
    df["MA20"] = df["Close"].rolling(20).mean()
    df["MA60"] = df["Close"].rolling(60).mean()

    return df

# -----
# 3. RSI
# -----

def calculate_rsi(
    df: pd.DataFrame,
    period: int = 14
) -> pd.DataFrame:
```



```

df = df.copy()

df["VOLUME_MA20"] = (
    df["Volume"]
    .rolling(20)
    .mean()
)

df["VOLUME_RATIO"] = (
    df["Volume"]
    / df["VOLUME_MA20"]
)

return df

```

여기까지가 계산 영역입니다.

아직 AI도 없고 매수/매도 판단도 없습니다.

이게 상당히 중요합니다.

4. 이제 여러 함수를 하나로 묶습니다

같은 `stock_analyzer.py` 아래에 추가합니다.

Python

실행됨

```

def calculate_indicators(
    df: pd.DataFrame
) -> pd.DataFrame:

    df = calculate_moving_averages(df)

    df = calculate_rsi(df)

    df = calculate_macd(df)

    df = calculate_atr(df)

    df = calculate_volume(df)

    return df

```

이 함수는 일종의 파이프라인입니다.

OHLCV

↓

Moving Average

↓

RSI

↓

MACD

↓

ATR

↓

Volume

이 개념을 잘 기억해 두세요.

나중에 LangGraph를 배우면 바로 이것이:

Node → Node → Node → Node

구조로 발전합니다.

5. 이제 분석 함수를 만듭니다

숫자를 계산하는 것과 해석하는 것을 분리하겠습니다.

Python

실행됨

```
def analyze_technical(
    df: pd.DataFrame
) -> dict:

    latest = df.iloc[-1]

    result = {}

    # 이동평균 분석

    if (
        latest["Close"] > latest["MA20"]
        and latest["MA20"] > latest["MA60"]
    ):
        result["trend"] = "bullish"
```

```

elif (
    latest["Close"] < latest["MA20"]
    and latest["MA20"] < latest["MA60"]
):
    result["trend"] = "bearish"

else:
    result["trend"] = "neutral"

# RSI 분석

if latest["RSI"] >= 70:
    result["rsi_signal"] = "overbought"

elif latest["RSI"] <= 30:
    result["rsi_signal"] = "oversold"

else:
    result["rsi_signal"] = "neutral"

# MACD 분석

if latest["MACD"] > latest["MACD_SIGNAL"]:
    result["macd_signal"] = "bullish"

else:
    result["macd_signal"] = "bearish"

# 거래량 분석

if latest["VOLUME_RATIO"] >= 1.5:
    result["volume_signal"] = "high"

else:
    result["volume_signal"] = "normal"

return result

```

이제 중요한 차이가 생겼습니다.

첫 번째 함수들은:

계산

을 했고,

이번 함수는:

을 합니다.

Agentic AI에서는 이 둘을 가능하면 분리하는 것이 좋습니다.

6. 점수 시스템을 만들어봅시다

예를 들어 100점 만점으로 만들어보겠습니다.

Python

실행됨

```
def calculate_technical_score(
    df: pd.DataFrame
) -> int:

    latest = df.iloc[-1]

    score = 50

    # Trend

    if (
        latest["Close"] > latest["MA20"]
        and latest["MA20"] > latest["MA60"]
    ):
        score += 20

    elif (
        latest["Close"] < latest["MA20"]
        and latest["MA20"] < latest["MA60"]
    ):
        score -= 20

    # RSI

    if 50 <= latest["RSI"] <= 70:
        score += 10

    elif latest["RSI"] >= 75:
        score -= 10

    # MACD

    if latest["MACD"] > latest["MACD_SIGNAL"]:
```

```

        score += 15

    else:
        score -= 15

    # Volume

    if latest["VOLUME_RATIO"] >= 1.5:
        score += 5

    # 0~100 제한

    score = max(
        0,
        min(100, score)
    )

    return score

```

예를 들어 결과가:

Technical Score = 78

라고 나올 수 있습니다.

7. BUY / HOLD / SELL까지 만들어 봅시다

Python

실행됨

```

def make_trading_signal(
    score: int
) -> str:

    if score >= 70:

        return "BUY"

    elif score <= 30:

        return "SELL"

    else:

        return "HOLD"

```

구조가 매우 단순합니다.

```
score >= 70
```

↓

```
BUY
```

```
30 < score < 70
```

↓

```
HOLD
```

```
score <= 30
```

↓

```
SELL
```

중요한 것은 지금 이 전략이 좋은 전략이냐가 아닙니다.

Python 함수들을 이용해서 판단 시스템을 만드는 구조를 이해하는 것이 목적입니다.

8. 모든 것을 하나의 함수로 묶습니다

Python

실행됨

```
def analyze_stock(
    ticker: str
) -> dict:

    df = get_stock_data(
        ticker
    )

    df = calculate_indicators(
        df
    )

    technical = analyze_technical(
        df
    )

    score = calculate_technical_score(
        df
    )

    signal = make_trading_signal(
        score
    )

    latest = df.iloc[-1]
```

```

result = {

    "ticker": ticker,

    "price": float(
        latest["Close"]
    ),

    "technical_score": score,

    "signal": signal,

    "analysis": technical,

    "indicators": {

        "MA5": float(
            latest["MA5"]
        ),

        "MA20": float(
            latest["MA20"]
        ),

        "MA60": float(
            latest["MA60"]
        ),

        "RSI": float(
            latest["RSI"]
        ),

        "MACD": float(
            latest["MACD"]
        ),

        "ATR": float(
            latest["ATR"]
        ),

        "volume_ratio": float(
            latest["VOLUME_RATIO"]
        )
    }
}

return result

```

여기서 아주 중요한 변화가 하나 있습니다.

최종 출력이 **dict**입니다.

Python

실행됨

```
{
    "ticker": "...",
    "price": ...,
    "technical_score": ...,
    "signal": "...",
}
```

Agentic AI에서는 이런 구조화된 출력이 매우 중요합니다.

나중에 LLM이 읽기도 쉽고 LangGraph State에 넣기도 쉽기 때문입니다.

9. `main.py`

이제 실행 프로그램을 만듭니다.

삼성전자의 Yahoo Finance ticker는:

005930.KS

입니다.

Python

실행됨

```
from stock_analyzer import analyze_stock

ticker = "005930.KS"

result = analyze_stock(
    ticker
)

print(
    result
)
```

실행합니다.

PowerShell

```
uv run python main.py
```

대략 이런 식의 결과가 나옵니다.

Python

실행됨

```
{
  'ticker': '005930.KS',

  'price': 85000,

  'technical_score': 80,

  'signal': 'BUY',

  'analysis': {

    'trend': 'bullish',

    'rsi_signal': 'neutral',

    'macd_signal': 'bullish',

    'volume_signal': 'normal'
  },

  'indicators': {

    'MA5': 84300,

    'MA20': 82100,

    'MA60': 79000,

    'RSI': 61.5,

    'MACD': 1240,

    'ATR': 2200,

    'volume_ratio': 1.23
  }
}
```

숫자는 예시입니다.

우리가 사실 무엇을 만든 것인가

겉으로 보면 단순한 주식 분석 프로그램인데, 구조적으로 보면 이미 **Agentic AI 시스템**의 뼈대입니다.

```
get_stock_data()

↓

calculate_indicators()

↓

analyze_technical()

↓

calculate_technical_score()

↓

make_trading_signal()
```

나중에는 이것이 이렇게 변합니다.

```
Market Data Tool
↓
Technical Agent
↓
Fundamental Agent
↓
Supply Agent
↓
ML Agent
↓
Risk Agent
↓
Decision Agent
```

따라서 **LangGraph**를 먼저 공부하는 것이 아니라 이런 **Python** 함수들이 어떻게 서로 데이터를 주고받는지 이해하는 것이 먼저입니다.

이번 단계에서 꼭 이해해야 할 5가지

이번 코드를 그냥 복사해서 실행하는 것보다 아래 개념을 이해하는 것이 더 중요합니다.

개념	현재 코드	나중의 Agentic AI
Input	ticker	Agent State
Function	<code>calculate_rsi()</code>	Tool

개념	현재 코드	나중의 Agentic AI
Pipeline	<code>calculate_indicators()</code>	Graph
Result	dict	Structured Output
Decision	<code>make_trading_signal()</code>	Decision Agent

특히:

Python

실행됨

```
def calculate_rsi(df):
```

라는 함수 하나가 향후:

```
LLM
↓
calculate_rsi Tool
↓
Python
↓
결과 반환
```

로 바뀝니다.

첫 번째 숙제

여기까지 코드를 직접 한번 실행해 보시는 걸 권합니다.

그 다음에는 1번 학습을 한 단계 더 진전시켜서 현재 분석기에 **ADX**와 **DI+/DI-**를 추가하겠습니다.

그러면 기술적 분석기는:

```
MA
RSI
MACD
ATR
ADX
DI+
DI-
Volume
```

을 모두 포함하게 됩니다.

그리고 그 다음에 바로 중요한 것을 해보겠습니다.

각 지표를 따로 계산하는 Python 함수를 왜 만들었는가?

를 설명하면서, 이 함수들 중 하나를 2단계인 LLM Tool Calling으로 변환해 보겠습니다.

즉 현재의:

Python

실행됨

```
calculate_rsi()
```

를 다음 단계에서는 실제로 LLM이 스스로 호출하도록 만들 겁니다.

그 순간부터 일반 Python 프로그램 → Agentic AI 프로그램으로 넘어가기 시작합니다.